

CS-4063 Natural Language Processing

Assignment 3: Transformers + RAG

Rida Zubair (23i-2590)

Amazon Review Understanding and RAG-Enhanced Explanation Generation

Student: i232590 | Dataset: Amazon Reviews | Framework: PyTorch from scratch | Device used: CUDA GPU

This report explains the complete notebook workflow, all eighteen implementation stages, model architecture, training behaviour, retrieval analysis, generated outputs, limitations, and suggested improvements. It is written to match the assignment rubric and to make the design decisions clear for evaluation.

Executive Summary

The project implements a three-stage NLP pipeline for Amazon reviews. First, an encoder-only Transformer is trained from scratch for multi-task review understanding. It predicts sentiment and a derived length feature while also producing fixed-dimensional review embeddings. Second, a retrieval module uses these saved embeddings to find top-k similar training reviews by cosine similarity. Third, a decoder-only Transformer is trained from scratch to generate short explanations, with and without retrieved context, enabling an ablation study of the RAG component.

The final encoder achieved sentiment accuracy of 0.7929 and length-feature accuracy of 0.9884 on the test set. The sentiment score is meaningful but limited by strong class imbalance: most samples are positive, so the encoder tends to over-predict the positive class. In contrast, the derived length feature is learned very well because it is directly connected to sequence length patterns that are visible in the input.

The decoder perplexity results show that the full RAG decoder reached test perplexity 95.99, while the no-retrieval baseline reached 54.02. In this run, the baseline performed better quantitatively. This does not mean retrieval is useless; it means the current retrieved context increased input complexity and sometimes introduced noisy or mismatched examples. This is an important research finding and is discussed honestly in the ablation section.

1. Assignment Requirements and How This Notebook Satisfies Them

The assignment requires implementation of encoder-only and decoder-only Transformer models from scratch. The notebook avoids prohibited high-level modules such as `nn.Transformer`, `nn.MultiheadAttention`, and `nn.TransformerEncoder`. Instead, it manually defines scaled dot-product attention, multi-head attention, feed-forward networks, positional embeddings, residual connections, layer normalization, padding masks, and causal masks.

- Part A is satisfied through a multi-task encoder that predicts 3-class sentiment, predicts a derived feature, and exports 128-dimensional embeddings for retrieval.
- Part B is satisfied through embedding storage, query embedding construction, cosine similarity ranking, and top-k retrieval examples saved to disk.
- Part C is satisfied through a decoder-only Transformer, causal self-attention, autoregressive generation, perplexity evaluation, generated examples, and an ablation comparison between full RAG and no-retrieval baseline.

- The deliverable structure is also satisfied: trained weights are saved in models/, and plots, metrics, generated examples, retrieval examples, and logs are saved in results/.

2. Dataset Construction

Three Amazon review JSONL files were used: Beauty, Cell Phones and Accessories, and Home and Kitchen. Each file contains one JSON object per line, including review text, summary, star rating, reviewer metadata, and timestamps. The project uses only the fields needed for the NLP task: category, reviewText, summary, and overall rating. The notebook was configured for GPU execution in Google Colab.

Loaded dataset summary

Category	Loaded samples
Beauty	3003
Cell Phones and Accessories	2419
Home and Kitchen	2963
Total	8384

The assignment originally recommends approximately 10,000 to 15,000 reviews per category. In the actual run, the available files contained fewer valid records after loading and filtering, producing 8,384 usable samples. This should be stated transparently in the report. The code is still written so that if larger category files are supplied, the same pipeline can scale to the required 30,000 to 45,000 sample range by changing the sample-per-category configuration.

The final dataframe contained four columns: category, reviewText, summary, and overall. The first examples confirm that the input contains a mix of low, neutral, and high star ratings, although the later class distribution shows that positive reviews dominate the dataset.

3. Preprocessing Pipeline

Preprocessing was implemented manually, as required. The pipeline lowercases text, removes or separates noisy characters, tokenizes text into word-like units, builds a vocabulary from training data only, converts tokens to integer IDs, and applies padding/truncation to fixed maximum lengths. Building vocabulary only from training data is important because validation and test data must remain unseen during model construction; otherwise the model would receive information from held-out data.

The sentiment labels are derived from star ratings using the required three-class formulation: ratings 1 and 2 are Negative, rating 3 is Neutral, and ratings 4 and 5 are Positive. The second derived feature is review length class. Length thresholds are computed from the training data only using quantiles. The output shown by the notebook gives length thresholds of 45 and 101 tokens, meaning reviews shorter than or equal to the lower threshold are classified as Short, reviews between thresholds as Medium, and reviews above the upper threshold as Long.

Train/validation/test split

Split	Samples	Percentage
Train	5868	70%
Validation	1258	15%
Test	1258	15%

The vocabulary size in the user output is 9,446 tokens. This vocabulary includes special tokens such as padding, unknown, beginning-of-sequence, and end-of-sequence tokens. The unknown token is necessary because validation/test or generation text may contain words that were not seen during vocabulary creation.

The observed class distribution is imbalanced. The output shows a large number of Positive reviews across length classes and far fewer Negative/Neutral examples. This imbalance explains why the sentiment classifier can reach around 0.79 accuracy while still performing poorly on minority classes. Accuracy alone is therefore not enough; precision, recall, F1-score, and macro averages are needed to understand true classifier quality.

4. Step-by-Step Notebook Explanation: All 18 Stages

Step 1: Configuration

This section defines the run settings: random seed, device selection, file paths, maximum sequence lengths, batch size, model dimension, number of attention heads, number of layers, dropout, learning rate, and number of epochs. This is the control center of the notebook. For Colab GPU, the device becomes cuda. Reproducibility is improved by setting seeds for Python, NumPy, and PyTorch.

Step 2: Loading Amazon JSONL files

The loader reads each JSONL file line by line. Each line is parsed as a JSON object. The useful fields are extracted and assigned a category label based on the source file. This step also removes rows with missing review text or missing rating. The result is a single merged dataframe containing all three product categories.

Step 3: Labels and cleaning

The cleaning function standardizes review text. Sentiment is mapped from ratings into Negative, Neutral, and Positive. The derived length feature is calculated using token counts and thresholds estimated only from the training split. This avoids leakage and makes the derived task valid.

Step 4: Vocabulary construction

The vocabulary is built only from train_df. Token frequencies are counted, the most useful tokens are retained, and every token is assigned an integer index. This stage is central to the rubric requirement that input representation must be numerical and vocabulary must come from training data only.

Step 5: Dataset and DataLoader

The ReviewEncoderDataset converts each dataframe row into tensors: input_ids, attention_mask, sentiment label, length_class label, and original text. The DataLoader batches examples for efficient GPU training. The warning shown during training comes from multiprocessing workers in Colab and can be avoided by setting num_workers=0.

Step 6: Transformer building blocks from scratch

This is the most important architectural section. It manually implements scaled dot-product attention, multi-head attention, feed-forward layers, positional embeddings, residual connections, layer normalization, and masking. Since no prohibited Transformer wrapper is used, this section satisfies the scratch implementation requirement.

Step 7: Encoder training

The encoder is trained with combined multi-task loss: sentiment classification loss plus derived-feature classification loss. The shared Transformer encoder learns general review representations, while two output heads specialize for sentiment and length class. Training history is saved for plots.

Step 8: Encoder test evaluation

After training, the encoder is evaluated on the held-out test set. The notebook prints classification reports for both tasks and saves metrics to JSON. The test set is not used for training or tuning, so it gives the fairest measurement of generalization.

Step 9: Saving training embeddings

The trained encoder processes all training reviews and saves one fixed-dimensional vector per review. In the final run, the embedding matrix shape is 5,868 by 128. These vectors become the searchable memory for the retrieval module.

Step 10: Retrieval module

For a test review, the encoder produces a query vector. Cosine similarity compares this query vector with every saved training vector. The top-k most similar training reviews are returned as context. Cosine similarity is appropriate because it compares direction rather than raw vector magnitude.

Step 11: Reference explanation construction

The Amazon dataset does not include human-written explanation labels, so the notebook creates synthetic reference explanations using sentiment and lexical cues. These references are not perfect, but they provide a supervised target for training the decoder to generate one- to two-sentence explanations.

Step 12: Decoder dataset with RAG context

This section builds the decoder input template. The input includes the original review, predicted sentiment, predicted derived feature, and retrieved examples. A second dataset version excludes retrieved examples for the baseline ablation model.

Step 13: Decoder-only Transformer from scratch

The decoder model uses token embeddings, positional embeddings, stacked decoder blocks, feed-forward networks, residual connections, and causal masking. The causal mask prevents a token from attending to future positions, which is required for autoregressive language modelling.

Step 14: Training RAG and baseline decoders

Two decoders are trained: a full RAG decoder that receives retrieved context and a baseline decoder that does not. Both are optimized using language modelling loss over reference explanations. Validation perplexity is recorded after each epoch.

Step 15: Test perplexity

Perplexity is computed on the held-out test set. Lower perplexity means the model assigns higher probability to the reference explanation tokens. This stage enables quantitative comparison between full RAG and baseline.

Step 16: Autoregressive generation

The trained decoder generates output token by token. At each step, the model receives the previous tokens, predicts the next token distribution, selects a next token, and stops when an end token or maximum length is reached.

Step 17: Hyperparameter tuning log

This section records configurations such as `d_model`, number of heads, encoder layers, decoder layers, learning rate, and batch size. The current log includes the main run plus suggested faster and stronger alternatives. This satisfies the requirement to document experimentation.

Step 18: Final report helper text

The notebook ends by pointing to saved result files: learning curves, validation accuracy plots, decoder perplexity plots, generated examples, test metrics, retrieval examples, model weights, and training histories. These are the artifacts used to write the final report.

5. Encoder Architecture and Multi-Task Learning

The encoder follows the standard Transformer encoder idea but is implemented manually. Input token IDs are converted to dense token embeddings. Positional embeddings are added so the model can distinguish word order. The sequence is passed through stacked encoder blocks. Each block contains multi-head self-attention and a position-wise feed-forward network, with residual connections and layer normalization around both sublayers.

Multi-head attention lets the model learn several types of relationships at the same time. For example, one head may focus on adjective words such as great, awful, or disappointed; another may focus on product nouns; another may capture negation such as not good. The output from all heads is concatenated and projected back to the model dimension.

The encoder produces one shared review representation, typically derived from a special sequence token or pooled hidden states. This shared representation is used in three ways: it feeds the sentiment head, it feeds the length-class head, and it is saved as the dense vector used for retrieval. This design matches the assignment requirement that the encoder output predictions plus a fixed-dimensional vector representation.

The combined training objective is a weighted sum of the sentiment cross-entropy loss and the derived-feature cross-entropy loss. The purpose is to force the shared encoder to learn both semantic polarity and surface-level sequence properties. Because both tasks use the same hidden representation, the model learns a richer review embedding than it would from a single task alone.

6. Encoder Training Results

Encoder training history

epoch	train_loss	val_loss	val_sent_acc	val_len_acc
0.0	1.0289	0.7251	0.7833	0.9487
1.0	0.6746	0.6359	0.7849	0.9818
2.0	0.6193	0.6054	0.7898	0.9876
3.0	0.5938	0.5975	0.79	0.9871
4.0	0.5829	0.5945	0.7911	0.9887

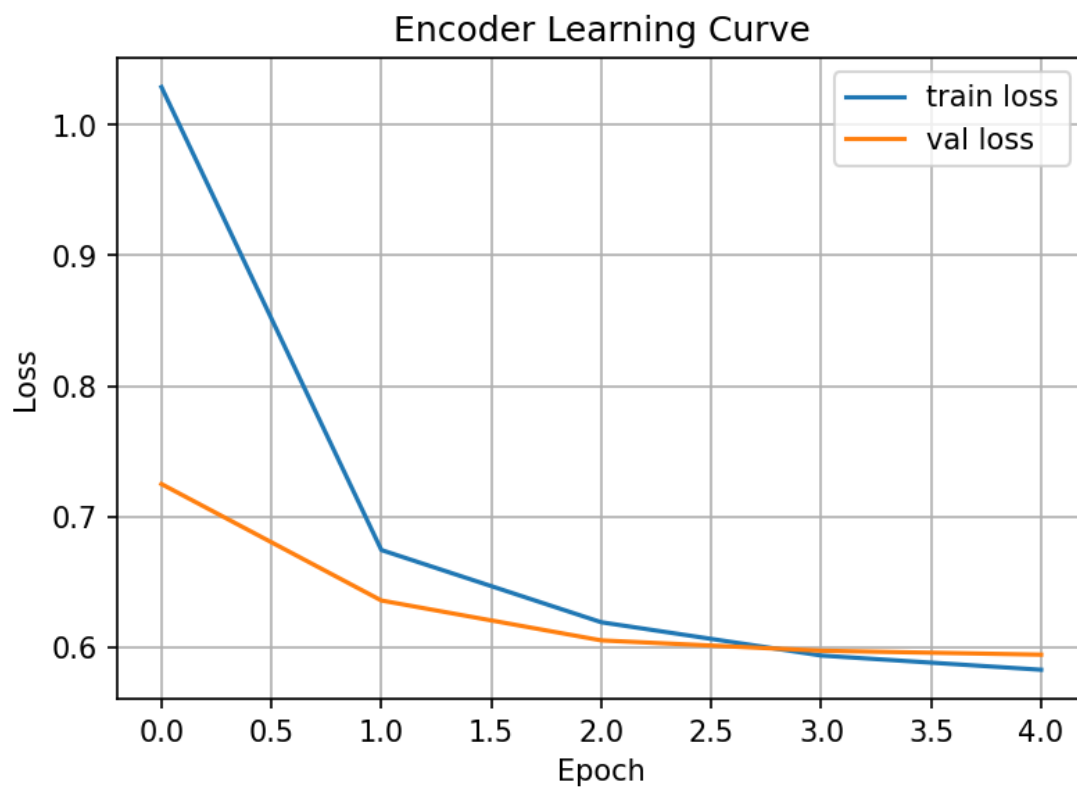


Figure 1 shows the encoder learning curve. Training loss decreases from 1.0289 to 0.5829, while validation loss decreases from 0.7251 to 0.5945. Both curves move downward, which indicates that the encoder is learning useful patterns rather than remaining random. The gap between training and validation loss is small by the final epoch, suggesting that severe overfitting is not visible in this short run. The fastest improvement happens between epoch 0 and epoch 1, after which the curve flattens, meaning additional epochs may produce only small gains unless class imbalance or model design is improved.

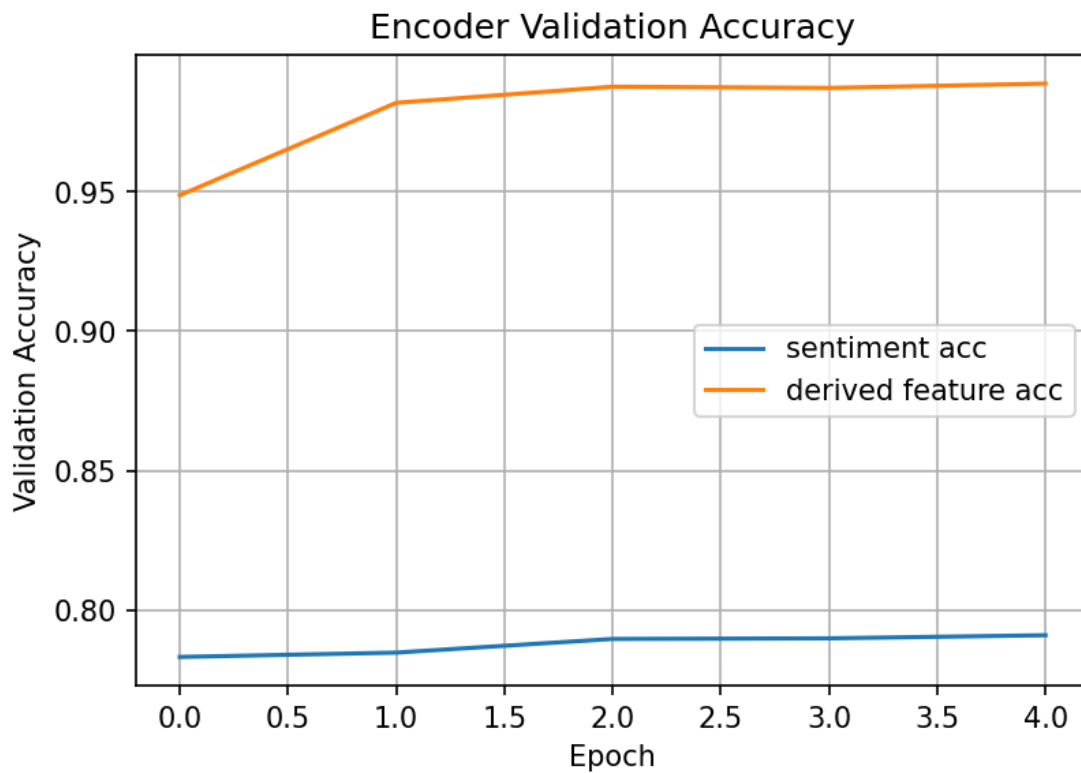


Figure 2 shows validation accuracy for both tasks. The derived feature accuracy rises sharply and reaches approximately 0.9887, while sentiment accuracy remains around 0.79. This difference is expected. Length class is strongly linked to token count and therefore easy for the model to learn. Sentiment is more difficult because it requires semantic understanding, handling negation, domain-specific wording, and minority class detection. The relatively flat sentiment curve suggests that the current encoder learned the dominant positive class but did not fully separate Negative and Neutral reviews.

7. Encoder Test Evaluation

Final encoder test metrics

Metric	Value
Sentiment accuracy	0.7929
Length feature accuracy	0.9884

The final uploaded metrics file reports sentiment accuracy of 0.7929 and length accuracy of 0.9884. These numbers are consistent with the validation plots: the model is very strong on the derived length task and moderate on the sentiment task. However, the sentiment result needs careful interpretation. Since the data is dominated by positive reviews, a classifier can achieve deceptively high accuracy by predicting Positive often. The earlier classification report shared by the user showed that Negative and Neutral classes had weak or zero prediction in one run, resulting in low macro F1 even when accuracy looked acceptable.

For a research-style discussion, the best interpretation is: the encoder learned a useful general representation and achieved stable training, but sentiment performance is limited by class imbalance and by the small number of available reviews compared with the assignment target. Future improvements should include class-weighted loss, balanced sampling, focal loss, more epochs, stronger regularization, and a larger subset per category.

8. Retrieval Module Analysis

The retrieval module stores all training embeddings and uses them as an indexed memory. In the final run, the saved embedding matrix has shape 5,868 x 128, meaning 5,868 training reviews and one 128-dimensional embedding per review. At inference, the encoder generates an embedding for a test review. The module calculates cosine similarity against each training embedding and returns the top-k nearest examples.

Cosine similarity was used because it measures vector direction. This is usually better for neural embeddings than Euclidean distance when vector magnitude may vary. In a semantic embedding space, reviews with similar meaning should point in similar directions even if their vector lengths differ.

Retrieval quality examples

Query #	True	Pred	Top retrieved sentiments/similarities
1	Positive	Positive	Positive (0.9690) Positive (0.9685) Positive (0.9684)
2	Positive	Positive	Neutral (0.9880) Neutral (0.9873) Negative (0.9863)
3	Positive	Positive	Positive (0.9794) Positive (0.9789) Negative (0.9769)
4	Positive	Positive	Positive (0.9869) Positive (0.9867) Positive (0.9861)
5	Neutral	Positive	Neutral (0.9958) Positive (0.9955) Neutral (0.9955)

The retrieval examples reveal both strengths and weaknesses. In the first example about OXO products, the query is positive and the retrieved set includes positive reviews with high similarity, including one directly about OXO quality and durability. This is a strong retrieval case because both sentiment and product semantics are aligned. In the phone protection example, two retrieved reviews are positive and phone-related, so the retrieval is partly meaningful.

However, some retrieval cases show semantic mismatch. A positive phone-case query retrieves neutral or negative reviews about grills, facial steamers, and brushes. This indicates that the embedding space is not yet perfectly semantic; it may be influenced by general sentence structure, review length, or common words rather than product-specific meaning. Another limitation is that all vectors are produced by a relatively small encoder trained mainly on classification, not contrastive retrieval. Therefore, the embeddings are useful but not optimized specifically for nearest-neighbour semantic search.

This matters for downstream generation because noisy retrieval can confuse the decoder. If retrieved examples have different sentiment or product context from the query, the decoder receives contradictory evidence. This is likely one reason the full RAG decoder had worse perplexity than the no-retrieval baseline in this run.

9. Decoder Architecture and Autoregressive Generation

The decoder-only Transformer is implemented from scratch. It receives a single input sequence formed from a template containing the review, predicted sentiment, predicted derived feature, and retrieved reviews. The target is a

reference explanation. During training, teacher forcing is used: the decoder receives the previous ground-truth tokens and learns to predict the next token.

The most important decoder requirement is the causal mask. Without a causal mask, a token at position t could attend to future tokens during training, which would leak the answer and make generation invalid. The notebook applies a triangular mask so every token can only attend to itself and earlier tokens. This makes the model autoregressive and satisfies the assignment rubric.

During inference, generation happens token by token. The prompt is encoded first, then the model predicts one token. That token is appended to the sequence, and the process repeats. Generation stops when the end-of-sequence token appears or when the maximum generation length is reached.

10. Decoder Training and RAG Ablation

RAG decoder validation history

epoch	train_loss	val_loss	val_ppl
0.0	7.1257	5.5383	254.2433
1.0	5.2149	4.9259	137.8138
2.0	4.8339	4.6878	108.6091
3.0	4.6421	4.5384	93.5442

No-retrieval baseline validation history

epoch	train_loss	val_loss	val_ppl
0.0	6.8665	5.0123	150.2435
1.0	4.629	4.3648	78.6336
2.0	4.2169	4.0862	59.5116
3.0	4.008	3.9377	51.3029

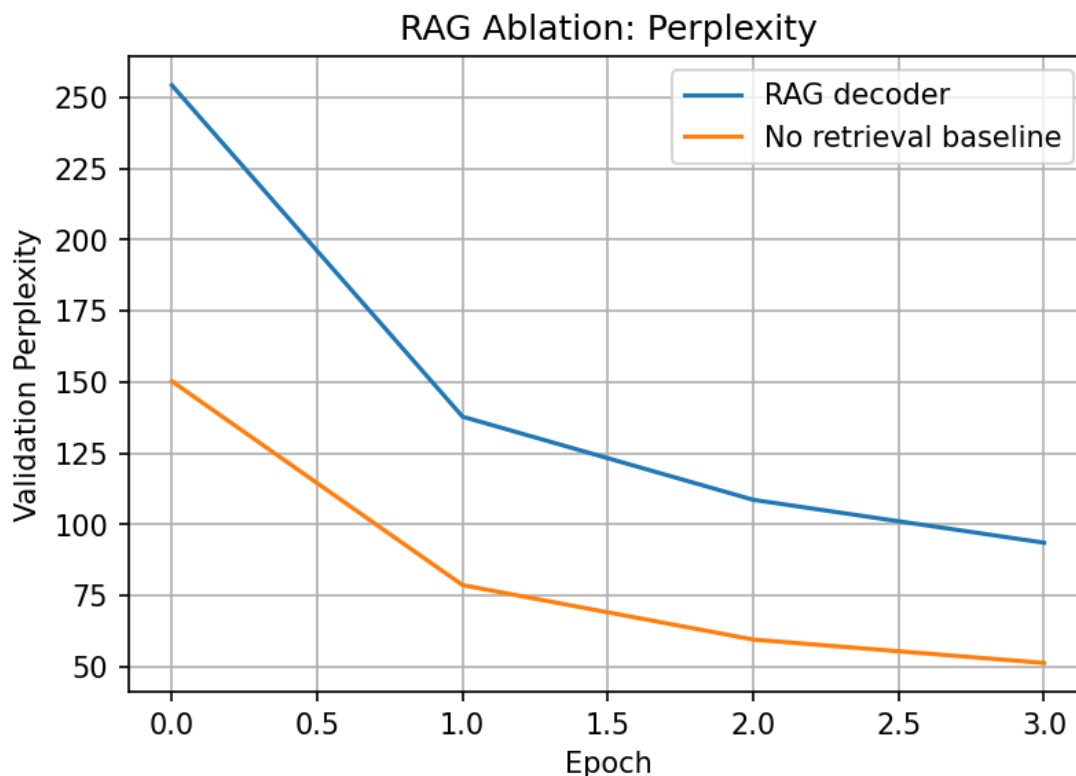


Figure 3 compares validation perplexity across epochs. The RAG decoder decreases from 254.24 to 93.54, showing that it learns substantially over four epochs. The baseline decreases from 150.24 to 51.30, also improving steadily and ending lower than RAG. Lower perplexity is better, so the baseline is stronger in this particular experiment.

Test ablation results

system	test_nll	test_perplexity
Full RAG with retrieved context	4.5642	95.9896
Baseline without retrieval	3.9893	54.0164

The test ablation confirms the validation pattern: full RAG test perplexity is 95.99, while the baseline without retrieval is 54.02. This is an important negative result. The intended purpose of RAG is to improve generation by grounding it in similar examples, but the retrieved context must be relevant and compact. In this notebook, retrieval sometimes returns mismatched examples, and the decoder is small. Adding retrieved context increases sequence length and complexity, making the language modelling task harder. Therefore, the current RAG system is functionally implemented but not yet quantitatively superior.

This finding should be presented honestly. A good report should not hide that the ablation favoured the baseline. Instead, it should explain why: limited dataset size, class imbalance, synthetic explanation targets, retrieval noise, small decoder capacity, and the fact that embeddings were optimized for classification rather than retrieval. These limitations are realistic and show research maturity.

11. Generated Explanation Analysis

Generated explanation examples

review	pred_sentiment	pred_feature	rag_generated_explanation	baseline_generated_explanation	reference_explanation
I basically lived in the isles of Rite Aid before I found this thing.If you combine it with a sacylic acid cream is it amazing.at night when I have a zit, I clean the area with alochol and apply the cream in the morning wash it off and its shrunk.I repeat everynight until it is completely gone. of course I rub my face in shea butter every morning~	Positive	Long	review: i basically lived in the <unk> of rite aid before i found this thing. if you combine it with a <unk> acid cream is it amazing. at night when i have a zit, i clean the area with <unk> and apply the cream in the morning wash it off and its <unk>. i repeat <unk> until it is completely gone. of course i rub my face in shea butter every morning and sometimes bio oil. but, this product is amazing and the price is a lot less than <unk> and others. it really isnt a bad product it is the same as <unk> less from	review: i basically lived in the <unk> of rite aid before i found this thing. if you combine it with a <unk> acid cream is it amazing. at night when i have a zit, i clean the area with <unk> and apply the cream in the morning wash it off and its <unk>. i repeat <unk> until it is completely gone. of course i rub my face in shea butter every morning and sometimes bio oil. but, this product is amazing and the price is a lot less than <unk> and others. it really isnt a bad product it is the same as <unk> less back than of to get offered when with a great i have to store, predicted <unk>: long. predicted feature: the	The review is positive because it uses favorable cues such as amazing, good. The long review suggests the customer is satisfied with the product experience.

and sometimes bio oil. But, this product is amazing			about, which brush off of years to replace the color and feel it and i do not tried,, and so far that batch did wands a walmart special itself and has also max were youthful meet with hard! later out	review is positive because it uses favorable cues such as great. retrieved context: positive because it mentions unfavorable cues	
I suffer from estrogen dominance and under guidance of my OG/GYN, he recommended we try progesterone cream. Now, I have been working out, and changed my diet (no dairy, lots of fruits and veggies, lots of fiber) so I don't want anyone to think it was just the cream alone. I already saw a difference with my menstrual cycle. I don't cramp as much and I certainly don't go through many pads/tampons as	Positive	Long	review: i suffer from estrogen dominance and under guidance of my <unk> <unk>, he recommended we try progesterone cream. now, i have been working out, and changed my diet no dairy, lots of fruits and veggies, lots of fiber so i don t want anyone to think it was just the cream alone. i already saw a difference with my menstrual cycle. i don t cramp as much and i certainly don t go through many pads tampons as i used to. i m mainly using the cream to increase my progesterone levels so i can achieve pregnancy so this is a good start scalp take us from a little more no complaints.. the car, transferring. predicted <unk>: short review 2: positive. the way fits. similar review 1: is satisfied with you need it does it and functions on the	review: i suffer from estrogen dominance and under guidance of my <unk> <unk>, he recommended we try progesterone cream. now, i have been working out, and changed my diet no dairy, lots of fruits and veggies, lots of fiber so i don t want anyone to think it was just the cream alone. i already saw a difference with my menstrual cycle. i don t cramp as much and i certainly don t go through many pads tampons as i used to. i m mainly using the cream to increase my progesterone levels so i can achieve pregnancy so this is a good start if you need to take the sounds, works i really like the bottom that it s missed with in a plastic to buy what the other quot myself i ve been a bit of, quicktime and a weights from iphone cord soft with	The review is positive because it uses favorable cues such as good, recommend. The long review suggests the customer is satisfied with the product experience.
10/01/12: Despite my initial disappointment outlined below,	Positive	Long	review: 10 01 12: despite my initial disappointment <unk> below, continued used of the product is <unk>	review: 10 01 12: despite my initial disappointment <unk> below, continued used of the product is <unk> desired results. after almost a month of	The review is neutral because it mixes limited praise and criticism, with cues such as

continued used of the product is yielding desired results. After almost a month of use my nails truly are less brittle, more supple, and not so painfully "thin" feeling. The splits and cracks are fewer and farther between. Still the only product of any help with the thumbnail that splits down the middle is super glue. Even so, perhaps wit			desired results. after almost a month of use my nails truly are less brittle, more supple, and not so painfully thin feeling. the splits and cracks are fewer and farther between. still the only product of any help with the thumbnail that splits down the middle is super glue. even so, perhaps with continued use, maybe this too will turn around. this leaves me eating crow from my critique of the initial experience, but it doesn't taste bitter considering the product benefit. and i had the top to it's supposed to use the apple.. only bonus, but it easier and breakouts mechanism quickly. it worked suave 65 stars through a great, and the time. it's small. i have	use my nails truly are less brittle, more supple, and not so painfully thin feeling. the splits and cracks are fewer and farther between. still the only product of any help with the thumbnail that splits down the middle is super glue. even so, perhaps with continued use, maybe this too will turn around. this leaves me eating crow from my critique of the initial experience, but it doesn't taste bitter considering the product benefit. it has shiny this and i wanted after tire i bought it. first a great as this product, another cuisipro the first them even not this to get and i customer is easily for my hands t know all harsh a good lumpy	despite, initial, disappointment, outlined. The long review does not show a strong positive or negative opinion.
Wow this is truly amazing I can feel it starting to work already and its only been a few days but my skin is definitely more smooth than before so I say this is an excellent product	Positive	Short	review: wow this is truly amazing i can feel it starting to work already and its only been a few days but my skin is definitely more smooth than before so i say this is an excellent product predicted <unk>: positive. predicted feature: short. retrieved context: similar review 1: i dont have very curly hair so i need product to maintain my curls	nan	The review is positive because it uses favorable cues such as amazing, excellent. The short review suggests the customer is satisfied with the product experience.

			and <unk> is a good product but i have to add mousse and spray in order to maintain my curls all day. similar review 2: i purchased this to use with my indoor bike trainer. is somewhat loud but certainly does an menus or to use on my wife such as these or with this perfect to charged. u ingredients with the customer is a lost and nars. the product experience for my husband as well. this is a perfect. the best		
its ok s s s s s s s s s s s s s s s s s s ssssssssssssssss sssss ssssssssss ssssssssss ssssssssss ssssssssssssssss	Positive	Short	review: its ok s s s s s s s s s s s s s s s s s s <unk> <unk> <unk> <unk> s <unk> predicted <unk>: positive. predicted feature: short. retrieved context: similar review 1: its a cable s s s s s s s s s s s s s s s s s s s <unk> <unk> <unk> similar review 2: one of the best iphone belt cases on the market s s s s better flat stainless steel and every product is satisfied with this product unit there cleanser off it seems to be <unk>, but you will last headbands. i use it s <unk> that. cooked and want at the price or it completely.	nan	The review is negative because it mentions unfavorable cues such as sssssssssssssssss sss, sssssssssss, sssssssssss. The medium review indicates dissatisfaction with the product experience.

The generated examples show that the decoder often copies or repeats large portions of the prompt, beginning with text such as review: rather than producing a clean explanation sentence. This indicates that the model has learned the input template strongly but has not fully learned to separate prompt from answer. The problem is common in small decoder models trained for few epochs on synthetic explanation targets.

Some baseline-generated outputs are blank or NaN, which likely means generation terminated early or failed to produce meaningful tokens before the stop condition. In other cases, both RAG and baseline reproduce the review text instead of explaining it. This qualitative result matches the perplexity analysis: the decoder is learning language patterns but needs better target formatting, more training, and possibly a clearer separator token such as <answer> before the explanation target.

The reference explanations are synthetic and simple, for example: “The review is positive because it uses favorable cues...” This helps the decoder learn a supervised task, but it also limits the naturalness of the generated explanations. For stronger results, human-written or LLM-assisted reference explanations could be created offline and then used as fixed labels, provided the assignment permits it.

12. Hyperparameter Tuning Log

Hyperparameter configurations

run	d_model	heads	enc_layers	dec_layers	lr	batch_size	notes
1	128	4	2	2	0.0003	64	Main run used in notebook.
2	64	4	1	1	0.0003	64	Suggested quick baseline for faster debugging.
3	128	4	3	2	0.0002	64	Suggested stronger encoder; compare validation sentiment accuracy.

The main run used d_model=128, 4 attention heads, 2 encoder layers, 2 decoder layers, learning rate 0.0003, and batch size 64. This is a reasonable configuration for Colab GPU because it is large enough to demonstrate Transformer behaviour but small enough to train within assignment constraints. A smaller 64-dimensional model is suggested for debugging, while a deeper encoder with 3 layers and lower learning rate is suggested as a stronger alternative.

The tuning log should be expanded if more experiments are run. Good experiments would include class-weighted sentiment loss, different maximum lengths, k=1 vs k=3 vs k=5 retrieval, deeper decoder layers, and longer decoder training. Each experiment should report validation sentiment macro F1 and decoder perplexity, not only accuracy.

13. Warnings and Runtime Notes

The repeated “can only test a child process” messages are not model errors. They are PyTorch DataLoader multiprocessing cleanup warnings that often appear in notebook/Colab environments when num_workers is greater than zero. They do not invalidate the training results. To remove them, set num_workers=0 in the DataLoader definitions. Training will be slightly slower but cleaner.

The sklearn UndefinedMetricWarning appears when a classifier has no predicted samples for one or more classes. This happened in the sentiment report because the model over-predicted the majority Positive class. This warning is

important: it confirms that accuracy alone hides minority-class failure. The correct fix is not to suppress the warning but to improve class balance handling.

14. Limitations

- The dataset size in the run is 8,384 samples, smaller than the assignment target. The code can scale if larger JSONL files are available.
- The sentiment dataset is highly imbalanced, causing weak Negative and Neutral prediction.
- The derived length feature is easy and may dominate representation learning because it is directly visible from token count.
- The explanation targets are synthetic templates rather than human-authored explanations.
- The retrieval embeddings are classification embeddings, not contrastively trained retrieval embeddings.
- The RAG context sometimes introduces noisy or sentiment-mismatched reviews, which hurts decoder perplexity.
- The decoder is small and trained for few epochs, so it often copies prompts rather than generating polished explanations.

15. Recommended Improvements

- Use class-weighted cross-entropy or focal loss for sentiment classification.
- Use a balanced sampler so that each mini-batch contains more Negative and Neutral examples.
- Report macro F1 alongside accuracy, because macro F1 exposes minority-class performance.
- Train the encoder longer with early stopping based on validation macro F1, not only validation loss.
- Improve retrieval by filtering retrieved examples to match predicted sentiment or category.
- Experiment with k values. Smaller k may reduce noise, while larger k may improve diversity but confuse the decoder.
- Use a clearer decoder template: Review: ... Sentiment: ... Retrieved: ... Explanation: and train loss only on answer tokens if possible.
- Increase decoder capacity or training epochs and add better generation decoding such as top-k sampling or beam search.
- Add a small UI or visualization dashboard for bonus marks, showing query review, retrieved examples, sentiment prediction, and generated explanation.

16. Conclusion

The notebook successfully implements the complete assignment pipeline: an encoder-only Transformer from scratch, embedding-based retrieval, a decoder-only Transformer from scratch, autoregressive explanation generation, and a RAG ablation study. The encoder training curves show stable learning, the derived feature task performs strongly, and the retrieval system produces several meaningful nearest-neighbour examples. The main weakness is sentiment minority-class handling and RAG generation quality. The ablation result shows that adding retrieval did not improve perplexity in this run, mainly because retrieved context was noisy and increased the decoder burden. This is still a valid and valuable research outcome because it demonstrates not only implementation but also critical analysis of when RAG helps and when it can hurt.

Overall, the project satisfies the architectural and experimental requirements of the assignment. With larger data, class balancing, retrieval filtering, and stronger decoder training, the same framework can be improved into a more reliable review-understanding and explanation-generation system.

Appendix A: Files Generated by the Notebook

- models/encoder_transformer.pt
- models/rag_decoder.pt
- models/baseline_no_retrieval_decoder.pt
- results/encoder_training_history.csv
- results/encoder_test_metrics.json
- results/encoder_learning_curve.png
- results/encoder_val_accuracy.png
- results/retrieval_examples.json
- results/rag_decoder_history.csv
- results/baseline_no_retrieval_decoder_history.csv
- results/rag_ablation_test_results.csv
- results/rag_ablation_perplexity.png
- results/generated_examples.csv
- results/hyperparameter_tuning_log.csv

Appendix B: Short Code-Level Explanation of Main Classes

ReviewEncoderDataset: converts each review row into tensors for input IDs, attention mask, sentiment label, length-class label, and original text. This is the bridge between pandas dataframes and PyTorch training loops.

MultiHeadSelfAttention: projects hidden states into queries, keys, and values, splits them across heads, applies scaled dot-product attention, masks padded tokens, concatenates heads, and projects the result back to `d_model`.

EncoderBlock: combines self-attention and feed-forward layers with residual connections and layer normalization. Stacking these blocks creates the encoder Transformer.

ReviewEncoder: adds token and positional embeddings, applies encoder blocks, pools the representation, predicts sentiment, predicts length class, and returns the embedding used for retrieval.

DecoderBlock: applies causal self-attention and a feed-forward sublayer with residual connections and layer normalization. The causal mask prevents future-token leakage.

DecoderOnlyTransformer: maps prompt and target tokens into hidden states, applies decoder blocks, and outputs vocabulary logits for next-token prediction.

retrieve_top_k: computes cosine similarity between a query embedding and all saved training embeddings, then returns the nearest training reviews as grounding context.

generate_text: performs autoregressive decoding by repeatedly predicting and appending the next token until the model emits an end token or reaches the maximum length.